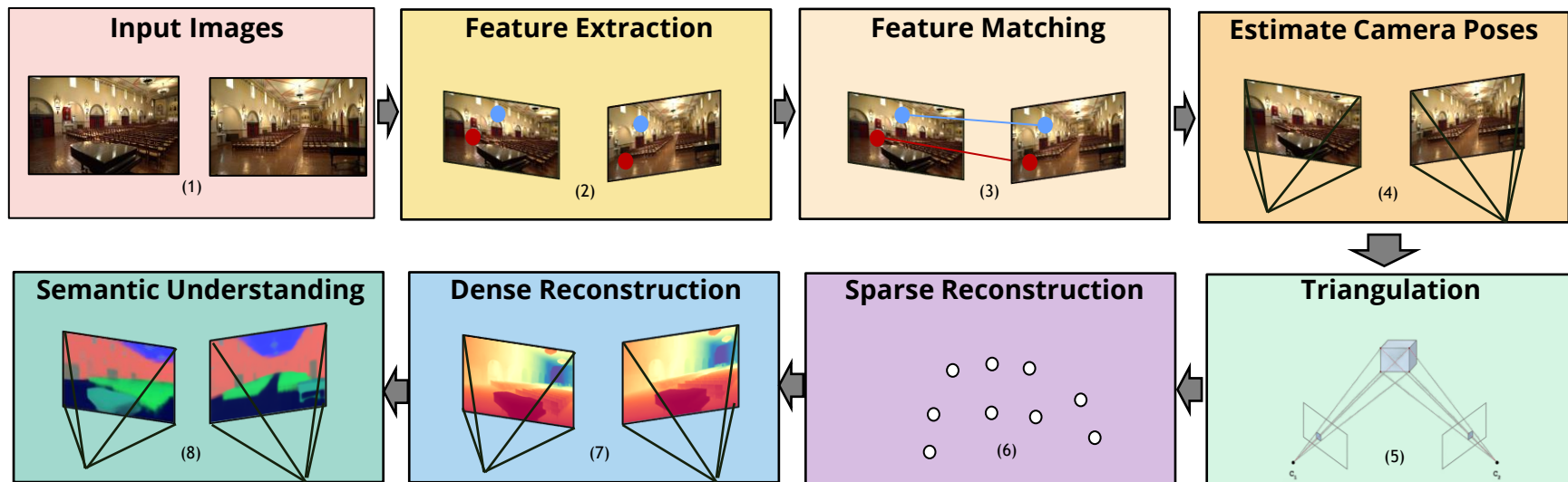


Today's Outline

- Neural Radiance Field and 3D Gaussian Splatting

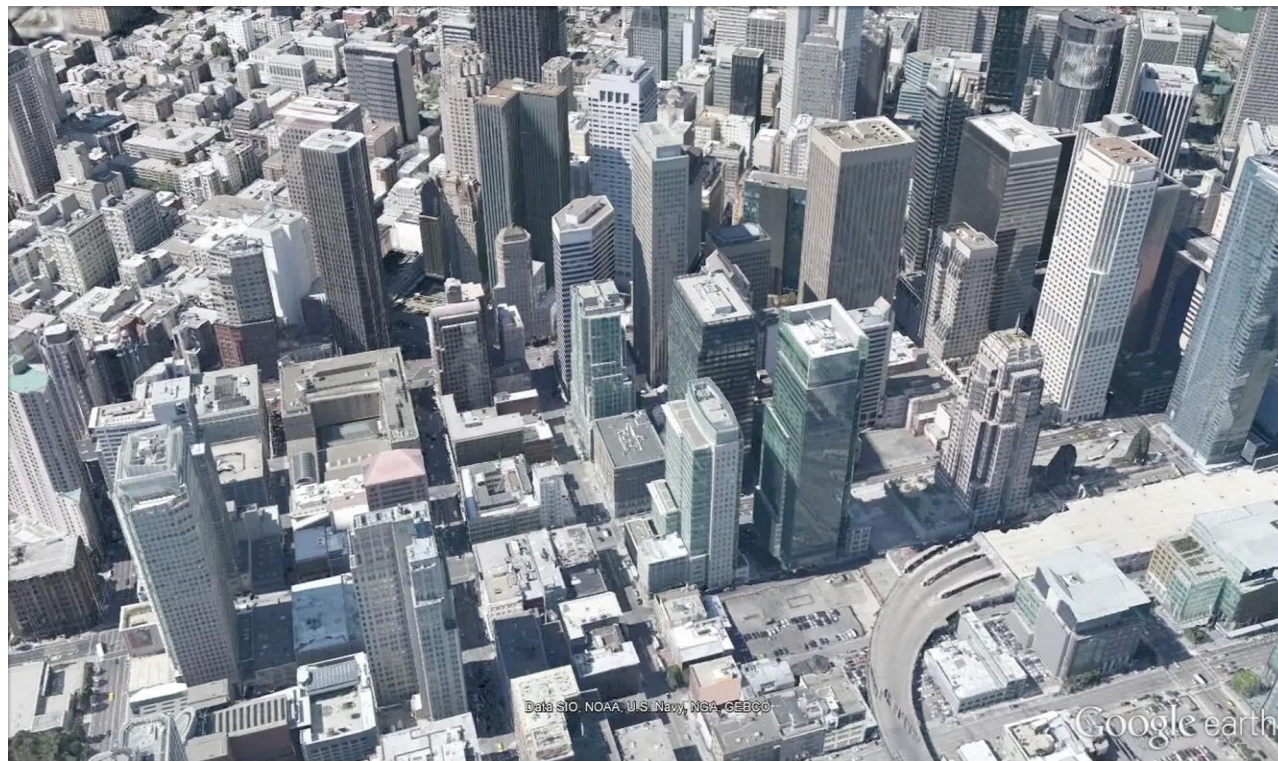
The Full 3D Perception and Reconstruction Pipeline



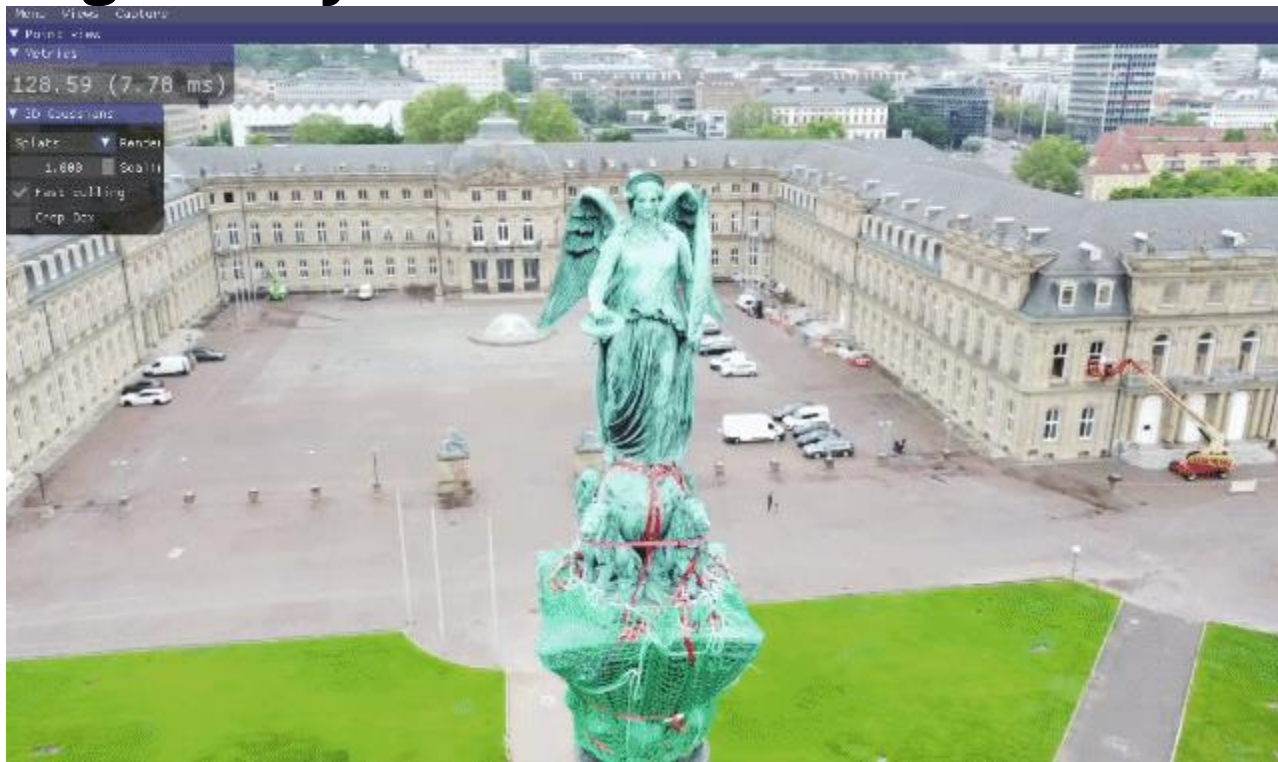
Capturing Reality – in 3D Point Cloud



Capturing Reality – in 3D Mesh



Capturing Reality



NeRF Can Render Highly Realistic Images



Input: sparsely sampled images

Output: new views of the same scene

NeRF Can Render Highly Realistic Images



Input Images



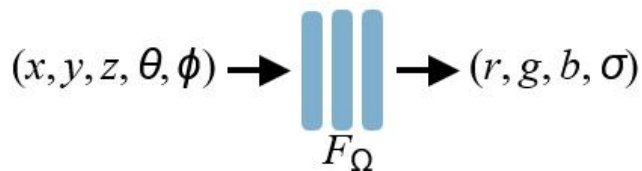
Optimization



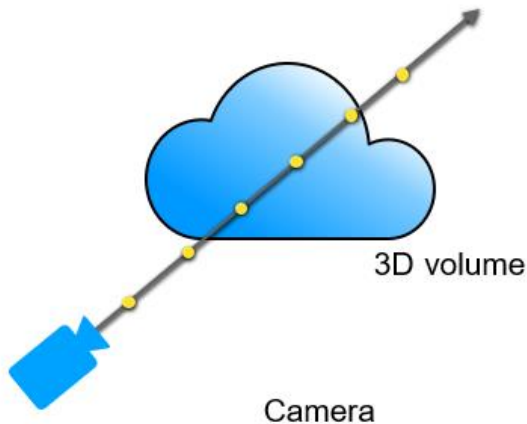
Rendering

Three Key Components

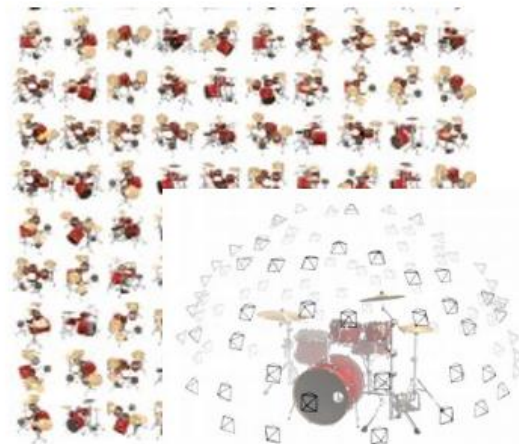
Objective: Synthesize
all training views



Neural Volumetric 3D
Scene Representation

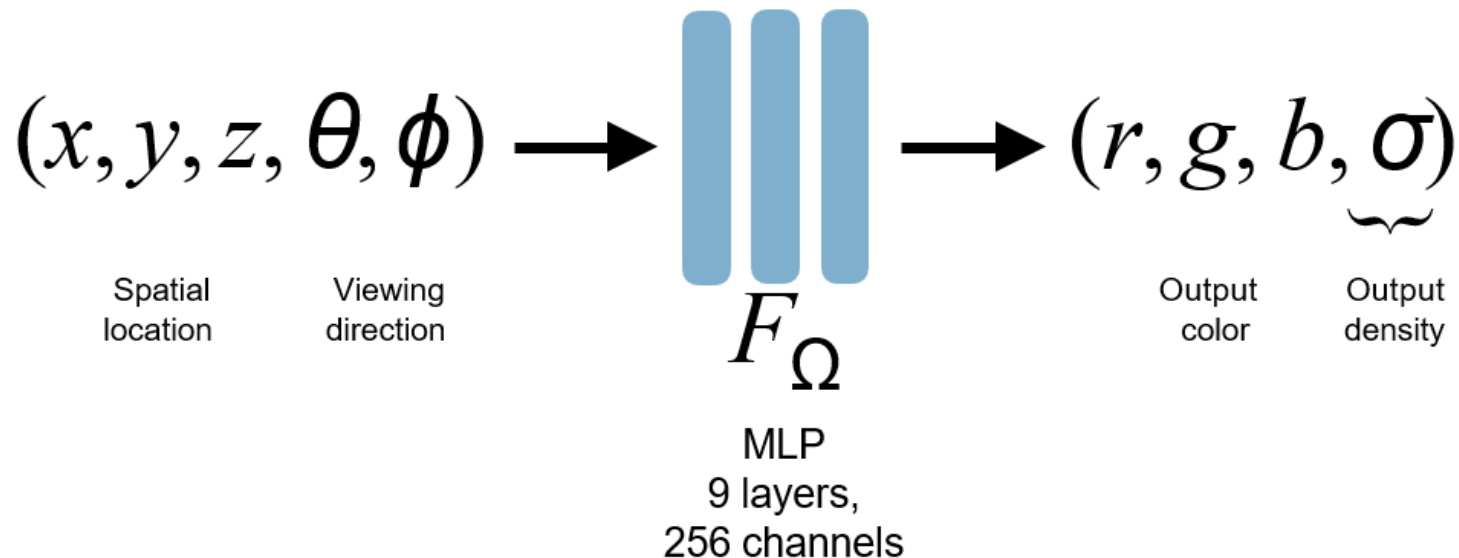


Differentiable Volumetric
Rendering Function



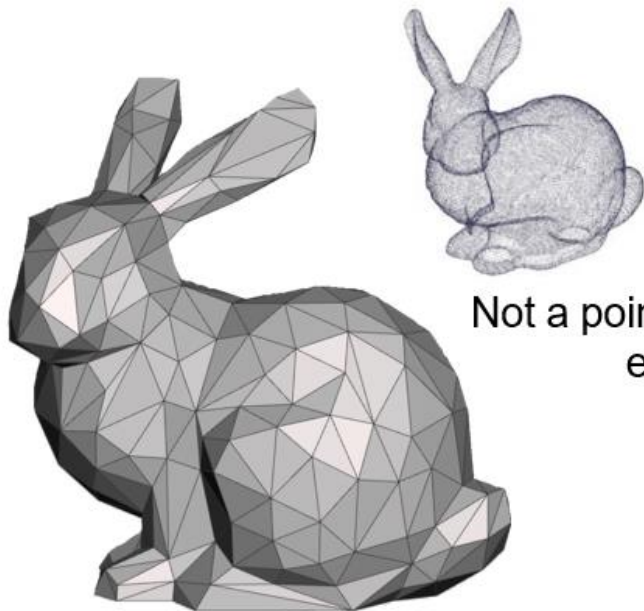
Optimization via
Analysis-by-Synthesis

Representing a 3D scene as a continuous 5D function



What kind of a 3D representation is this?

It is not a Mesh, not point cloud either



Not a point cloud
either



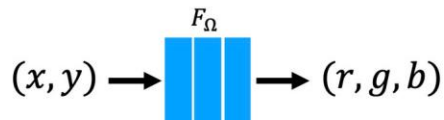
It is volumetric

It's continuous voxels made of shiny transparent cubes

A simple example

Part 1(b): 2D Image Fitting

Now, let's try to fit a 2D image with a multilayer perceptron (MLP)! In class we learn that we can store an 2D image with a coordinate-based MLP (as shown in the figure below). The input to this MLP is 2D pixel coordinate (x, y) as a pair of floating point numbers, and the output is RGB color of the corresponding pixel. This is a simple supervised learning problem, and we can just use simple gradient descent to train the network weights and see what happens.



First, let's define the network architecture for this 2D fitting task. We provide an example of network architecture called `Model2d` below. You can run all the way to the last cell in TODO 1(b) to execute the training process. Without any modification, you should get $\text{PSNR} \sim 27$ after training for 10,000 iterations.

Now, your task is to modify `Model2d`, such that **after training for 10,000 iterations with `num_encoding_functions=6`, PSNR is greater than or equal to 30**. Please do not change the model name, the name of the existing arguments, or the input/output dimensions. Hint: You can try different model structure (e.g. more/fewer layers, smaller/bigger hidden dimensions).

*PSNR is an image quality measurement. Higher PSNR generally indicates that the reconstruction is of higher quality.

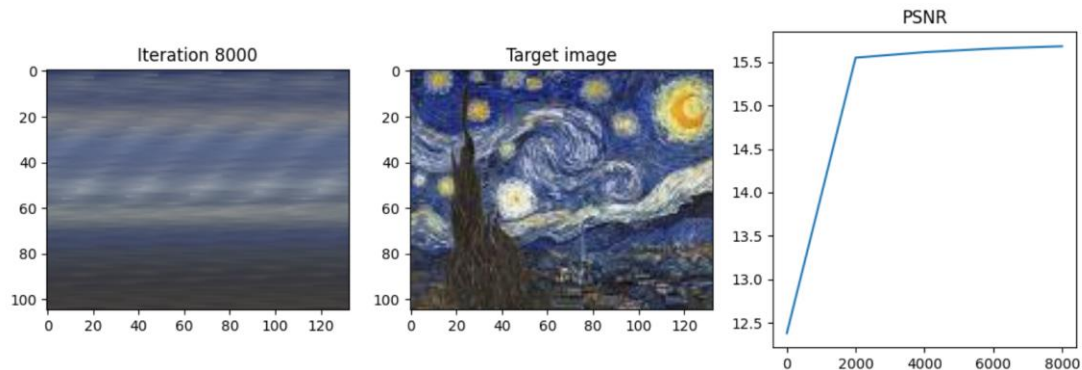
Training comparison w/ and w/o positional encoding

Run the following cell to initialize the training function.

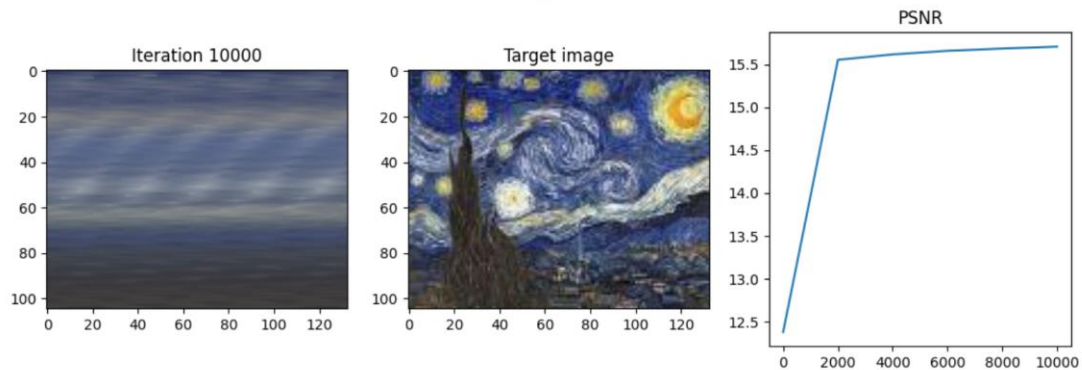
```
[4]: # Load painting image
painting = imageio.imread("Starry-Night-canvas-Vincent-van-Gogh-New-1889_12.jpg")
painting = torch.from_numpy(np.array(painting, dtype=np.float32)/255.).to(device)
height_painting, width_painting = painting.shape[:2]

plt.figure(figsize=(13, 4))
plt.title("Starry Night painting")
plt.imshow(painting.detach().cpu().numpy())
plt.show()
```

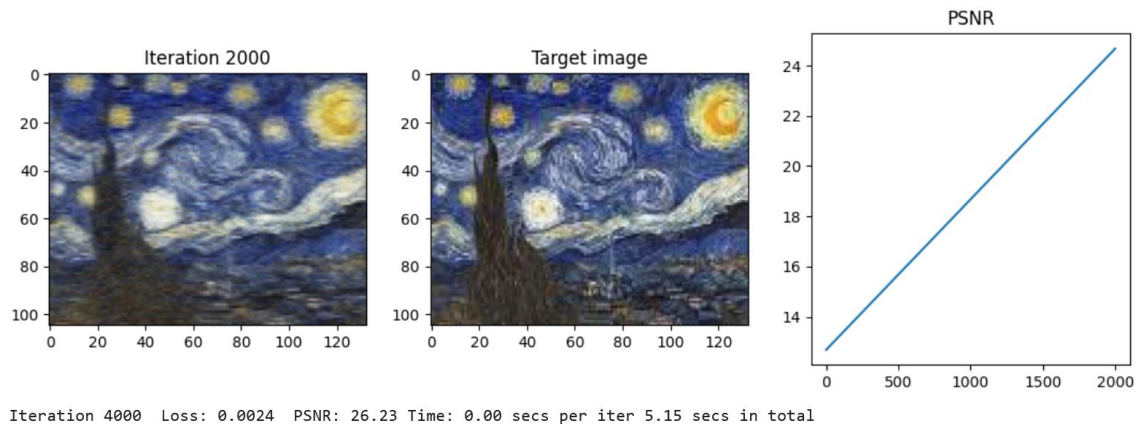
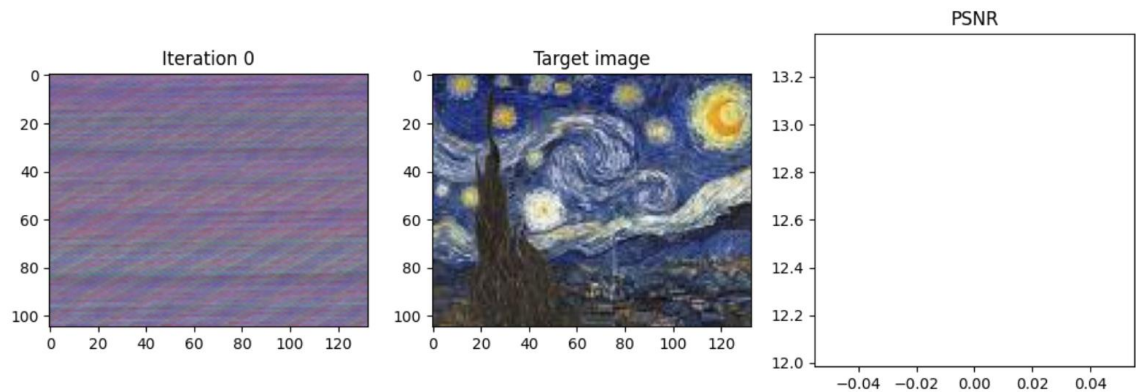
A simple example



Iteration 10000 Loss: 0.0269 PSNR: 15.70 Time: 0.00 secs per iter 12.67 secs in total

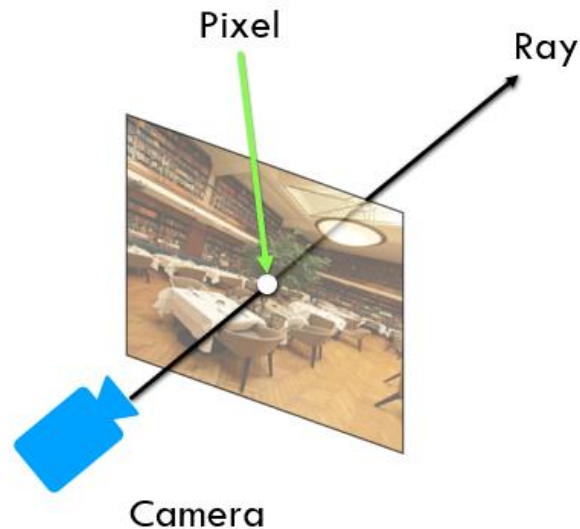


A simple example

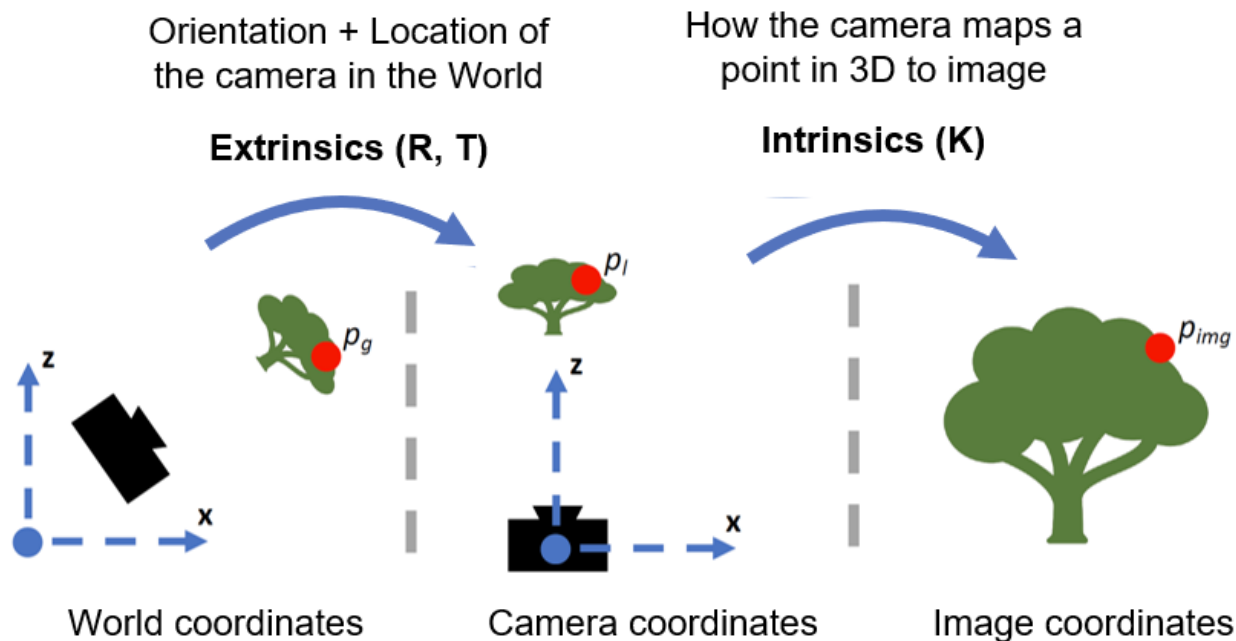


Cameras and rays

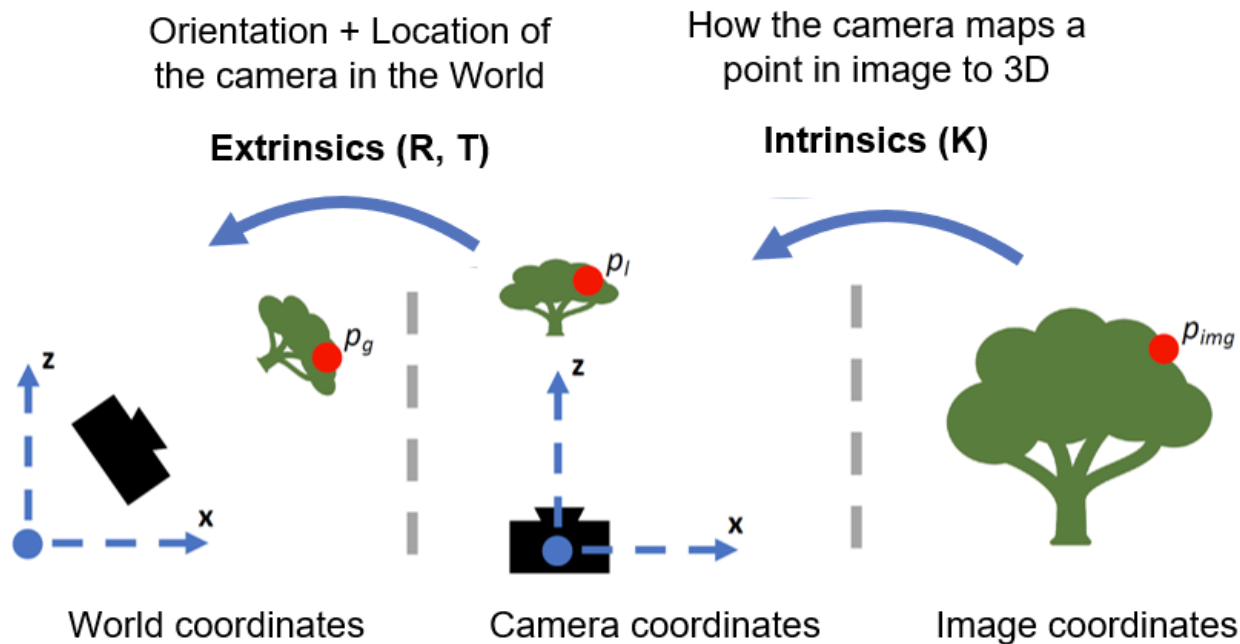
- We need the mathematical mapping from $(camera, pixel) \rightarrow ray$
- Then can abstract underlying problem as learning the function $ray \rightarrow color$ (the “plenoptic function”)



Coordinate frames + Transforms: world-to-camera



Coordinate frames + Transforms: world-to-camera

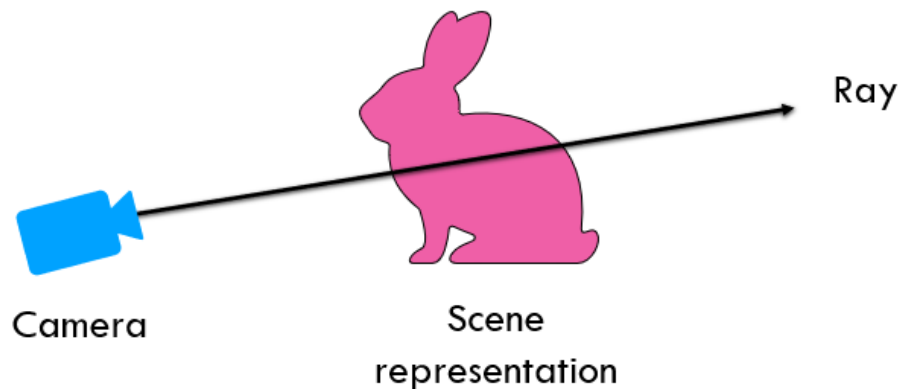


Neural Volumetric Rendering



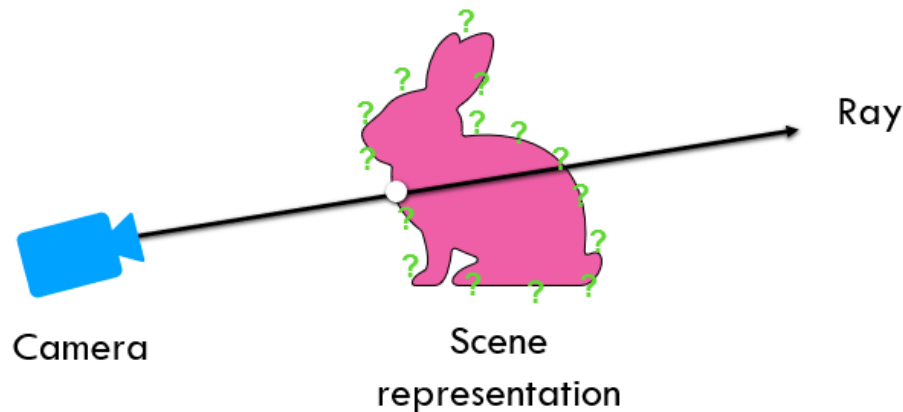
continuous, differentiable rendering model without
concrete ray/surface intersections

Surface vs. volume rendering



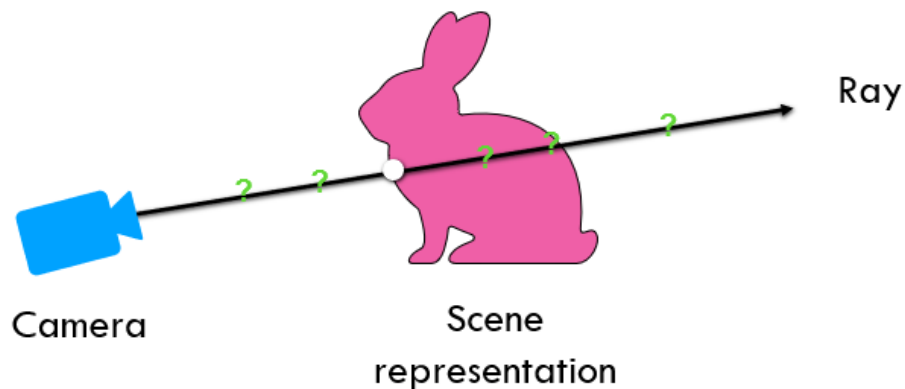
Want to know how ray interacts with scene

Surface vs. volume rendering



Surface rendering — loop over geometry, check for ray hits

Surface vs. volume rendering



Volume rendering – loop over ray points, query geometry

Volume Rendering: The Core of NeRF

Rendering model for ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

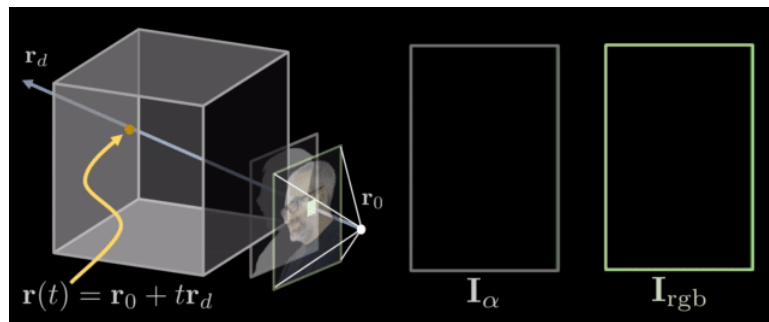
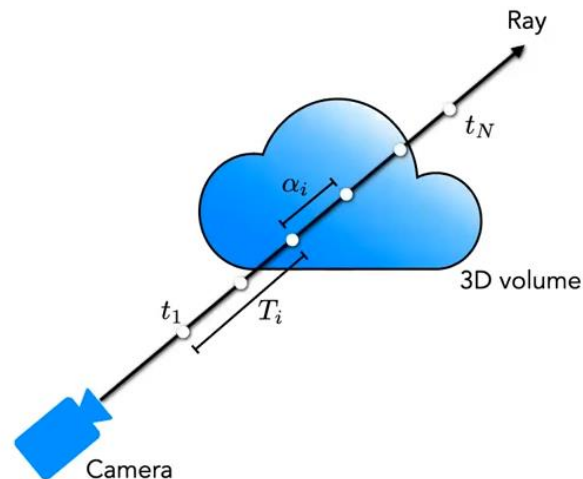
weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

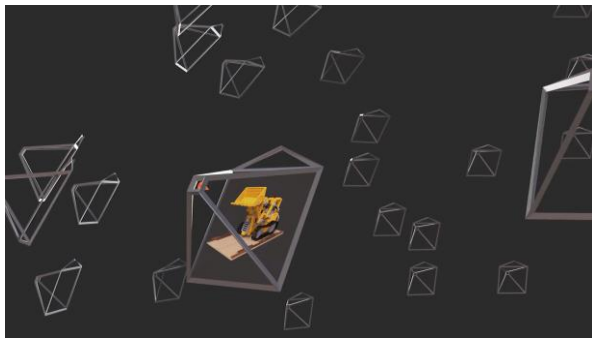
How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$

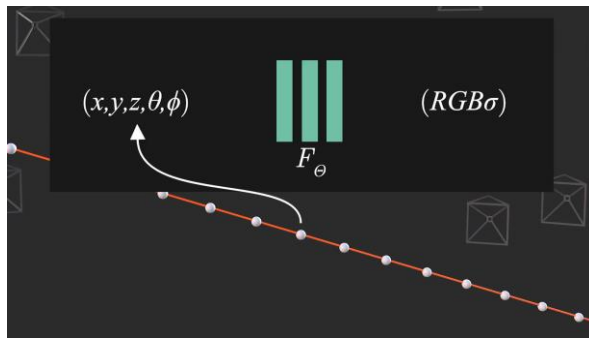


Rendering: Projecting 3D to 2D

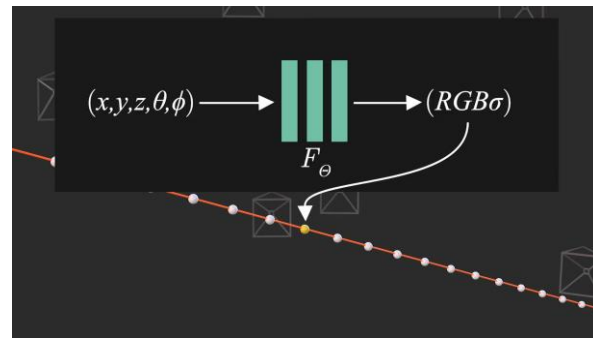
- Neural Radiance Field (NeRF) is a subset of Neural Rendering
 - Volumetric-based Rendering Pipelines
- Typically have 3 stages
- Let the pixels find what features they need



1). Mapping Pixel to Ray-Points

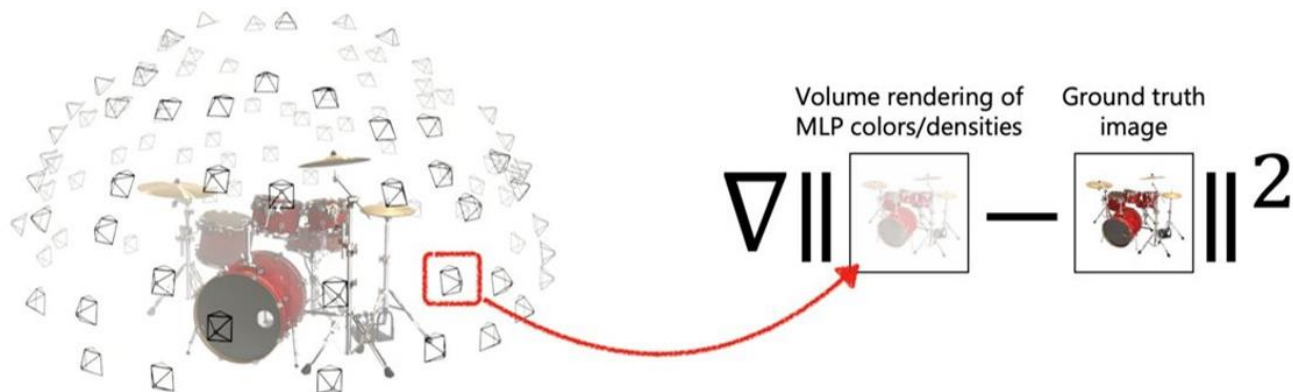


2). Get Points' Color



3). Accumulate for Pixel's color

Neural Radiance Field (NeRF)

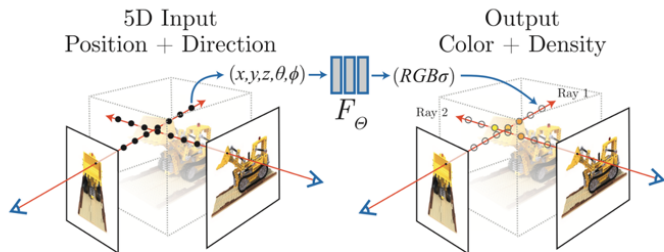


$$\arg \min_{\mathbf{V}} \sum_i \left\| \mathcal{R}(\mathbf{V}; \mathbf{R}^{(i)}, \mathbf{t}^{(i)}, \mathbf{P}^{(i)}) - \mathbf{y}^{(i)} \right\|_2^2$$

Neural Radiance Field (NeRF)

$$\arg \min_{\mathbf{V}} \sum_i \left\| \mathcal{R}(\mathbf{V}; \mathbf{R}^{(i)}, \mathbf{t}^{(i)}, \mathbf{P}^{(i)}) - \mathbf{y} \right\|_2^2$$

1. Neural radiance 3D scene representation



2. Volume rendering

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt, \text{ where } T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$$

Ray color

Transmittance

Density

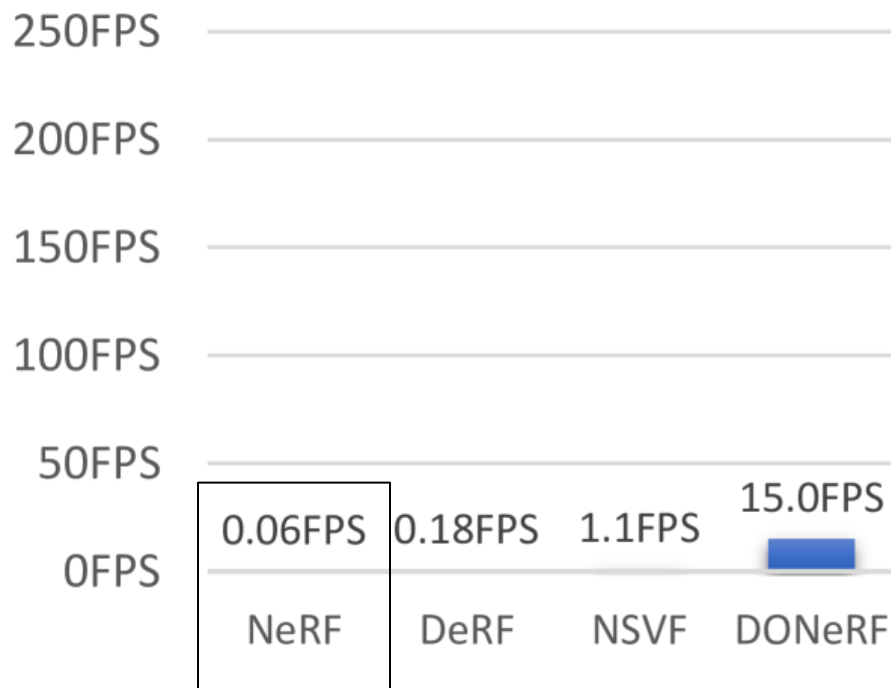
Color

$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$

Position

Direction

NeRF Is Too Slow for Real-Time Rendering



Can We Use an Explicit 3D Representation

WebGL 3D Gaussian Splat Viewer

By Kevin Kerl, Code on GitHub.

► Use mouse or arrow keys to navigate.



Can We Use an Explicit 3D Representation

3D Gaussian Splatting for Real-Time Radiance Field Rendering

BERNHARD KERBL*, Inria, Université Côte d'Azur, France

GEORGIOS KOPANAS*, Inria, Université Côte d'Azur, France

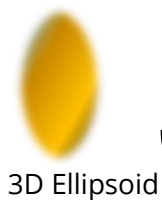
THOMAS LEIMKÜHLER, Max-Planck-Institut für Informatik, Germany

GEORGE DRETTAKIS, Inria, Université Côte d'Azur, France





Each Primitive



XYZ

Scale

Color

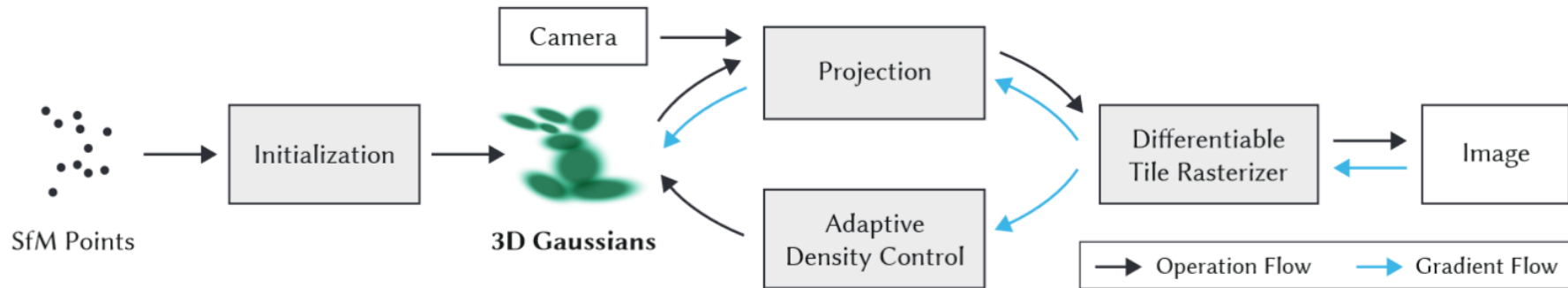
Rotation

Opacity

Geometry + **Appearance**



3DGS, the optimization



3DGS, the performance

Dataset Method Metric	Mip-NeRF360						Tanks&Temples						Deep Blending					
	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem
Plenoxels	0.626	23.08	0.463	25m49s	6.79	2.1GB	0.719	21.08	0.379	25m5s	13.0	2.3GB	0.795	23.06	0.510	27m49s	11.2	2.7GB
INGP-Base	0.671	25.30	0.371	5m37s	11.7	13MB	0.723	21.72	0.330	5m26s	17.1	13MB	0.797	23.62	0.423	6m31s	3.26	13MB
INGP-Big	0.699	25.59	0.331	7m30s	9.43	48MB	0.745	21.92	0.305	6m59s	14.4	48MB	0.817	24.96	0.390	8m	2.79	48MB
M-NeRF360	0.792 [†]	27.69 [†]	0.237 [†]	48h	0.06	8.6MB	0.759	22.22	0.257	48h	0.14	8.6MB	0.901	29.40	0.245	48h	0.09	8.6MB
Ours-7K	0.770	25.60	0.279	6m25s	160	523MB	0.767	21.20	0.280	6m55s	197	270MB	0.875	27.78	0.317	4m35s	172	386MB
Ours-30K	0.815	27.21	0.214	41m33s	134	734MB	0.841	23.14	0.183	26m54s	154	411MB	0.903	29.41	0.243	36m2s	137	676MB

Challenges with 3D-GS



Scene Bicycle



Model Size: 1400MB



Scene Garden



Model Size: 1400MB



Scene Stump



Model Size: 1100MB



Scene Truck

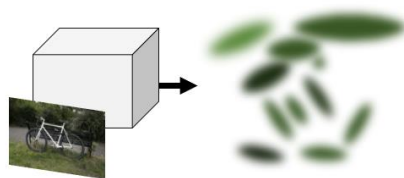


Model Size: 552MB

Significant amount of disk spaces are required in the explicit representation.

Poses significant scalability challenges and hindering the splatting efficiency.

Challenges with 3D-GS



Multi-view Images 3D Gaussians

Excessive Gaussian number



Often millions on each scene

Spherical Harmonic (SH) coefficient
requires high dimensions



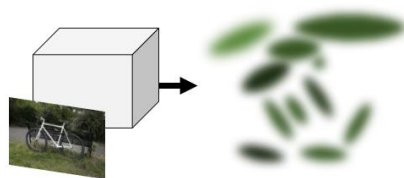
e.g., 3D-GS uses 3-degree SH, requires 48
dimension on each Gaussian

Inefficient storage



Similar attributes can be quantized
into lower bitwidth representation

Challenges with 3D-GS



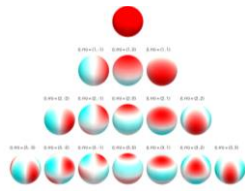
Multi-view Images 3D Gaussians

Excessive Gaussian number



Often million on each scene

Spherical Harmonic (SH) coefficient
requires high dimensions



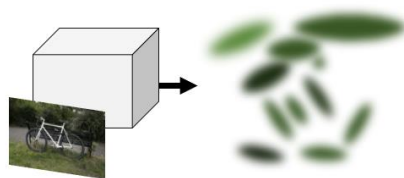
e.g., 3D-GS uses 3-degree SH, requires 48
dimension on each Gaussian

Inefficient storage



Similar attributes can be quantized
into lower bitwidth representation

Challenges with 3D-GS



Multi-view Images 3D Gaussians

Excessive Gaussian number



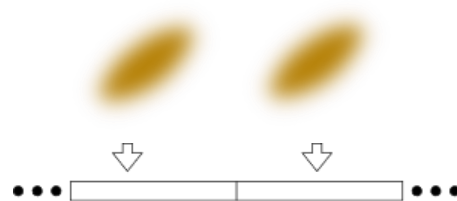
Often million on each scene

Spherical Harmonic (SH) coefficient requires high dimensions



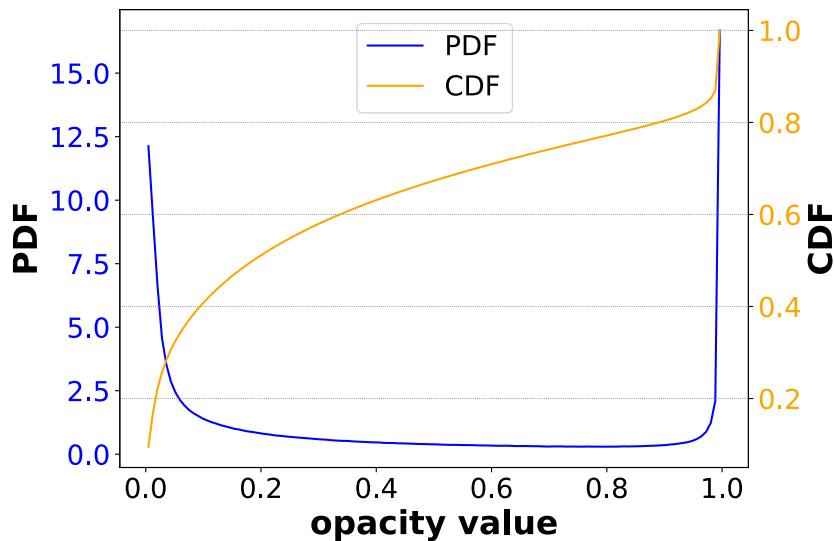
e.g., 3D-GS uses 3-degree SH, requires 48 dimension on each Gaussian

Inefficient storage



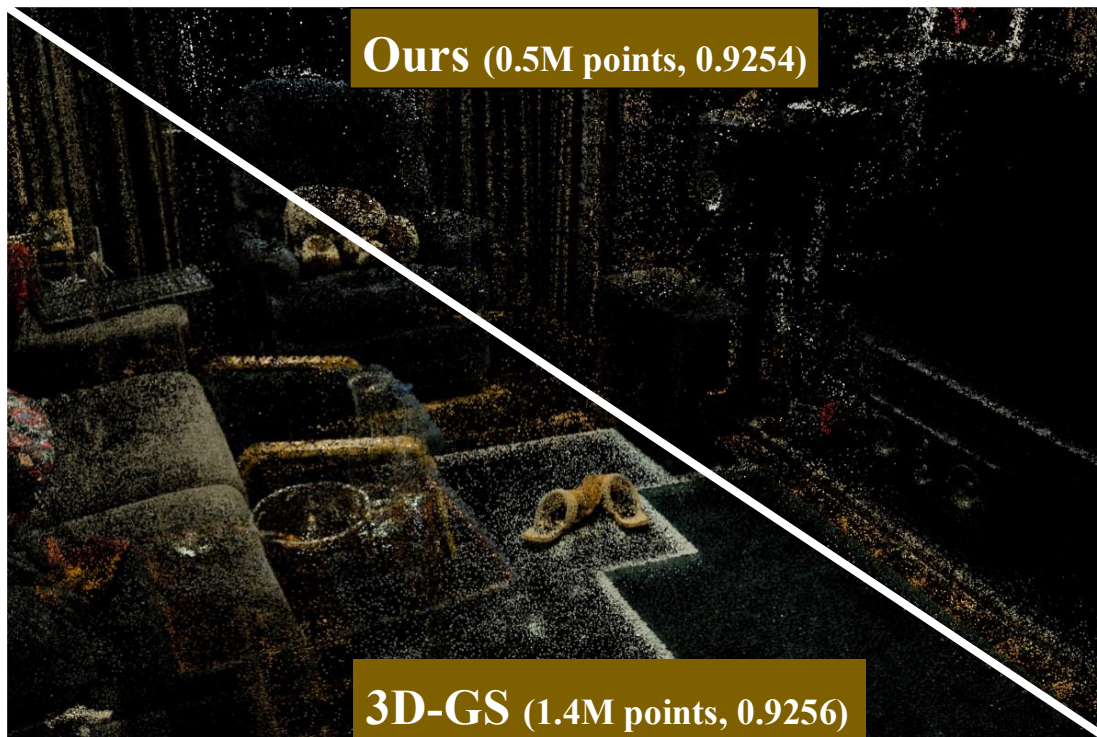
Similar attributes can be quantized into lower bandwidth representation

Excessive Gaussian Number



Simply using Gaussian Opacity as criteria results in significant performance drop, motivate us to find a better metric.

Excessive Gaussian Number



We propose to reduce the Gaussians via its global significance.

Excessive Gaussian Number



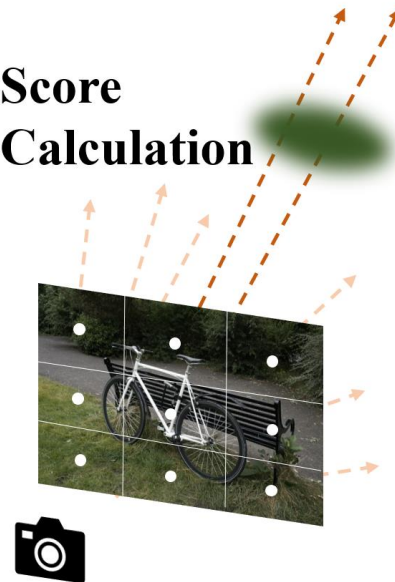
$$GS_j = \sum_{i=1}^{MHW} \mathbb{1}(G(\mathbf{X}_j), r_i) \cdot \sigma_j \cdot \gamma_j(\Sigma),$$



$$\gamma(\Sigma) = (V_{\text{norm}})^\beta,$$

$$V_{\text{norm}} = \min \left(\max \left(\frac{V(\Sigma)}{V_{\text{max90}}}, 0 \right), 1 \right).$$

**Score
Calculation**



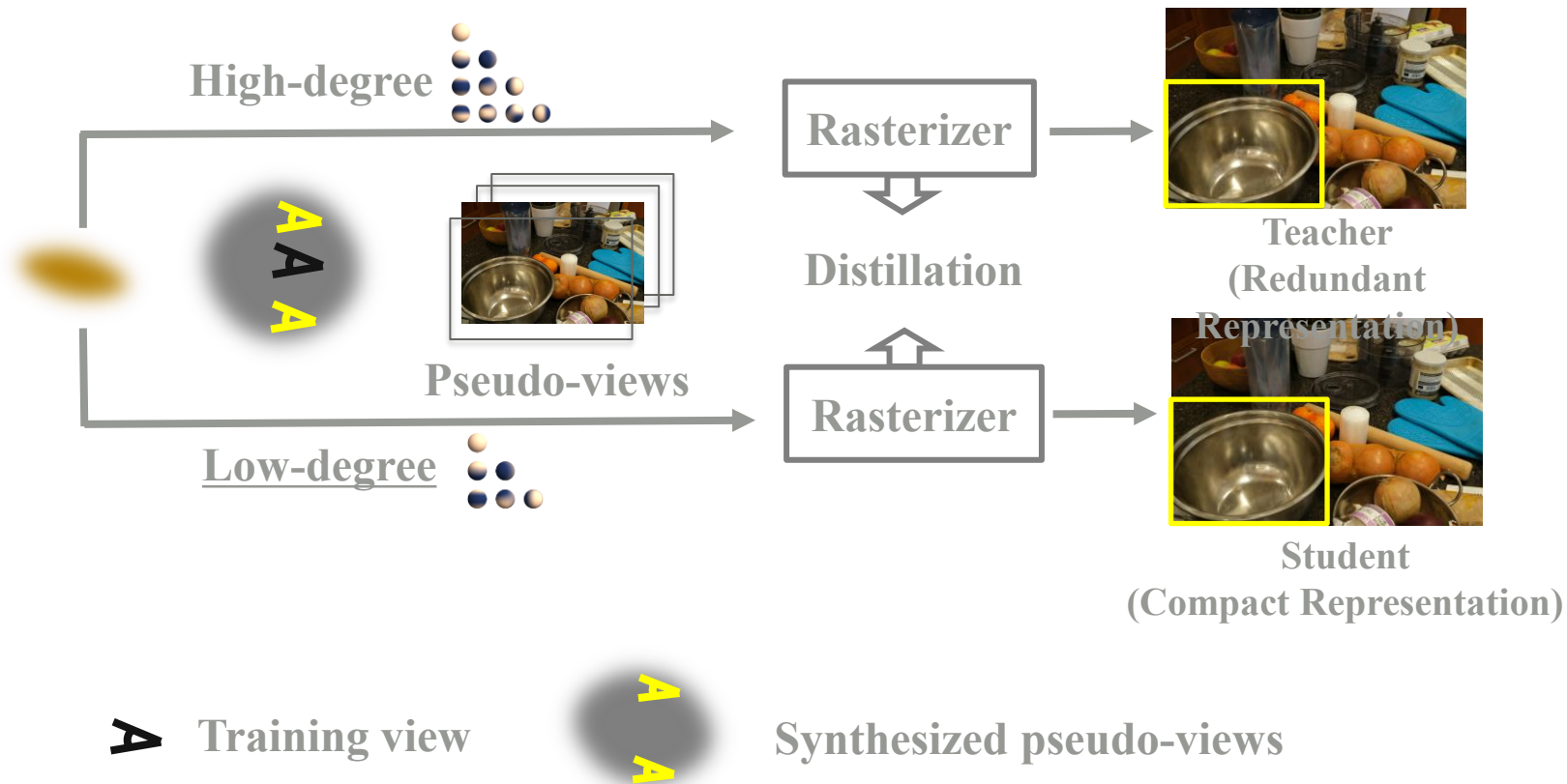
$G(\mathbf{X}_j)$: gaussian at location $\mathbf{X}_j$

$\mathbb{1}$: Hit indicator function

σ_j pacity

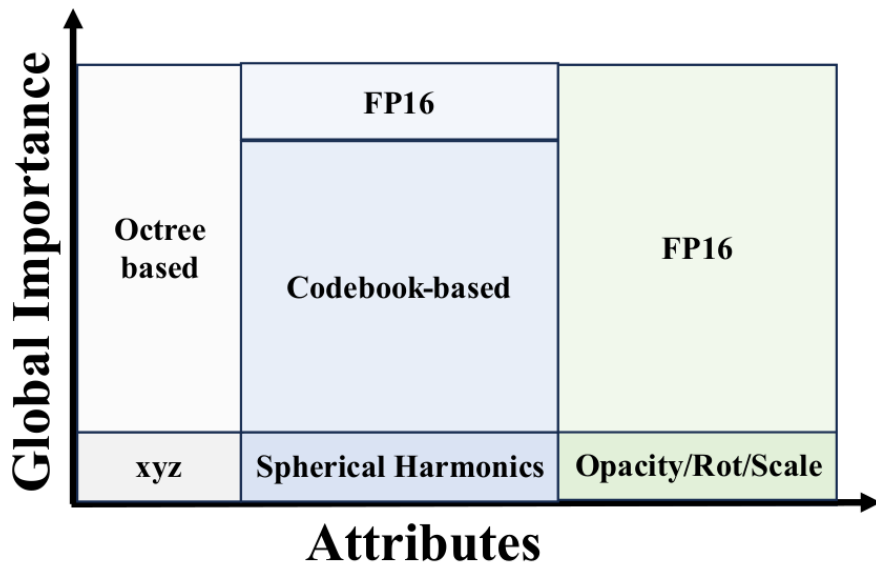
$\gamma(\Sigma)_{\text{ne}}$

Reduce the SH degree via Distillation



Vectree Quantization

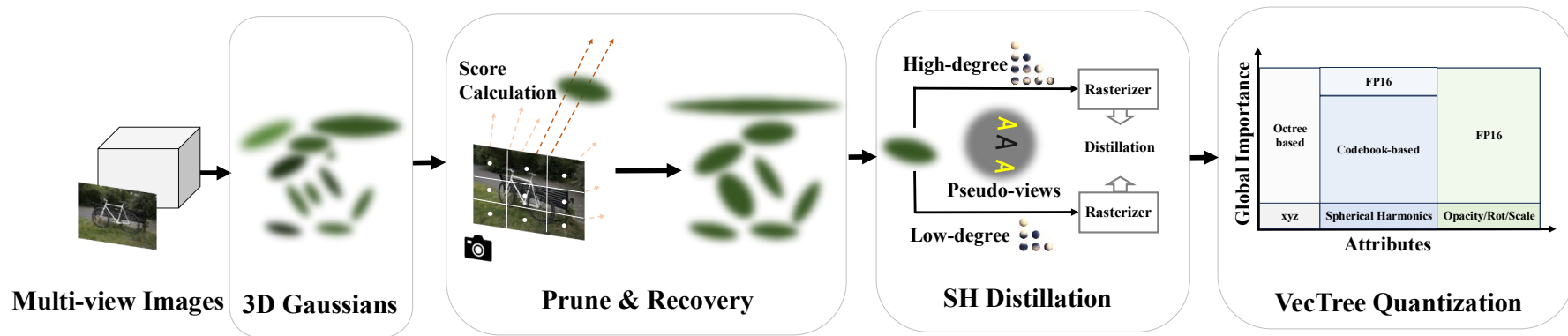
Color features can be represented by a compact codebook, while other attributes are sensitive for lossy compression.



Quantization and compression:

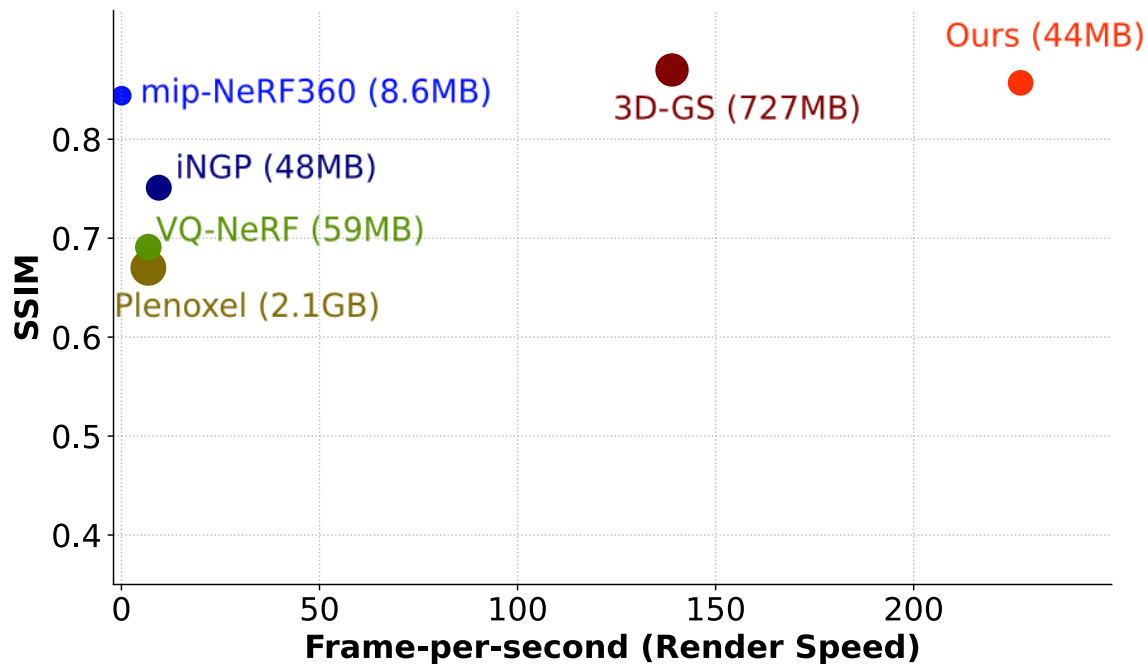
- Octree Compression:
 - Moderate compression ratio, and lossless.

LightGaussian, the Efficient Representation



- **Prune & Recovery:**
 - Reduce Gaussian number
- **SH Distillation:**
 - Distill SH coefficient into a more compact representation.
- **Vector Quantization:**
 - Hybrid compression different Gaussian attributes

Results



LightGaussian achieves 15x reduction in model size, and run at 200+FPS, thanks to the compact representation.

Results

Methods	Mip-NeRF 360 Datasets					Tank and Temple Datasets				
	FPS↑	Size↓	PSNR↑	SSIM↑	LPIPS↓	FPS↑	Size↓	PSNR↑	SSIM↑	LPIPS↓
Plenoxels [64]	6.79	2100MB	23.62	0.670	0.443	11.2	2700MB	21.07	0.719	0.379
INGP-Big [42]	9.43	48MB	26.75	0.751	0.299	2.79	48MB	21.92	0.744	0.305
Mip-NeRF 360 [5]	0.06	8.6MB	29.23	0.844	0.207	0.09	8.6MB	22.21	0.759	0.257
VQ-DVGO [36]	6.53	59.01MB	25.86	0.691	0.365	-	-	-	-	-
3D-GS [30]	139	727MB	29.13	0.870	0.185	106	380MB	23.11	0.822	0.219
Ours	215	44.36MB	28.45	0.857	0.210	209	23.53MB	22.83	0.807	0.242

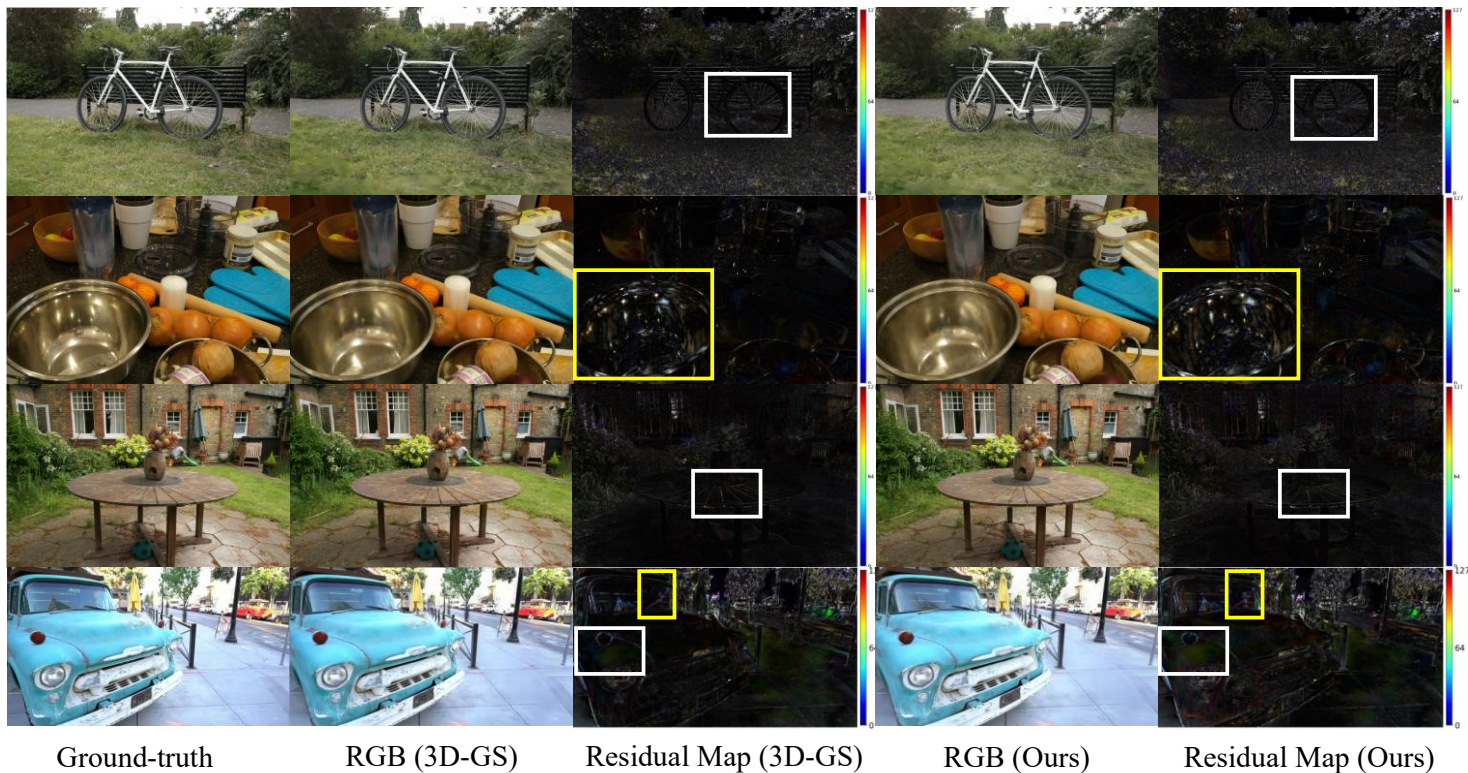
Quantitative Comparisons in Real-world Large-scale Scenes: LightGaussian (Ours) achieves the best balance between model size and accuracy and can run at a real-time speed.

Results

Exp#	Model	FPS $\uparrow$	Size $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
[1]	Baseline (3D-GS [30])	192.05	353MB	31.68	0.926	0.200
[2]	+ Gaussian Pruning.	312.30	116MB	30.32	0.911	0.222
[3]	+ Co-adaptation	303.99	116MB	31.85	0.925	0.206
[4]	+ SH Compactness.	318.97	77MB	30.54	0.914	0.217
[5]	+ Distillation	304.20	77MB	31.47	0.922	0.211
[6]	+ Pseudo-views	300.60	77MB	31.59	0.923	0.211
[7]	+ Codebook Quant.	300.60	23MB	31.26	0.917	0.220
[8]	+ Octree Compression.	300.60	20MB	31.26	0.917	0.220
[9]	LightGaussian (Ours)	300.60	20MB	31.26	0.917	0.220

- Gaussian Pruning[2] + Co-adaptation[3] reduces the Gaussian number, with trivial impact in quality.
- SH Compactness[4], along with distillation + pseudo-view augmentation preserve the specular reflection variance when changing viewpoint.
- VecTree Hybrid Compression [7][8] further compress the model with minimum quality decrease.

Results



We observe that LightGaussian preserves the specular reflections (yellow boxes) after converting to a compact format. Additionally, a slight lightness change is noted after the conversion (white boxes).

3D Gaussian Splatting [Kerbl et al. 2023] has shown state-of-the-art performance on real-time radiance field rendering.

However, it has no semantic scene understanding which significantly limits human-centric applications.



3D
Gaussians

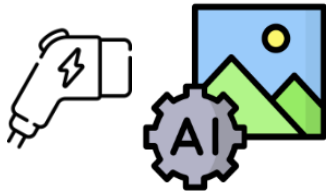


Radiance Field

In this work, we supercharge the 3D Gaussian Splatting to learn feature fields via knowledge distillation from 2D foundation models.



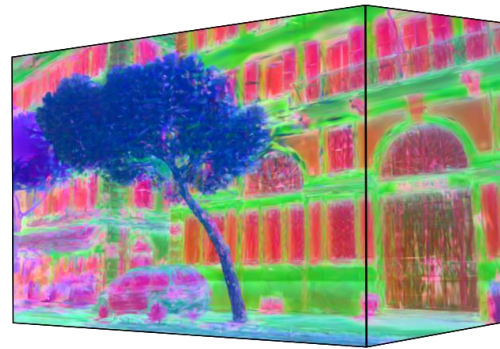
3D Gaussians



2D Foundation Model

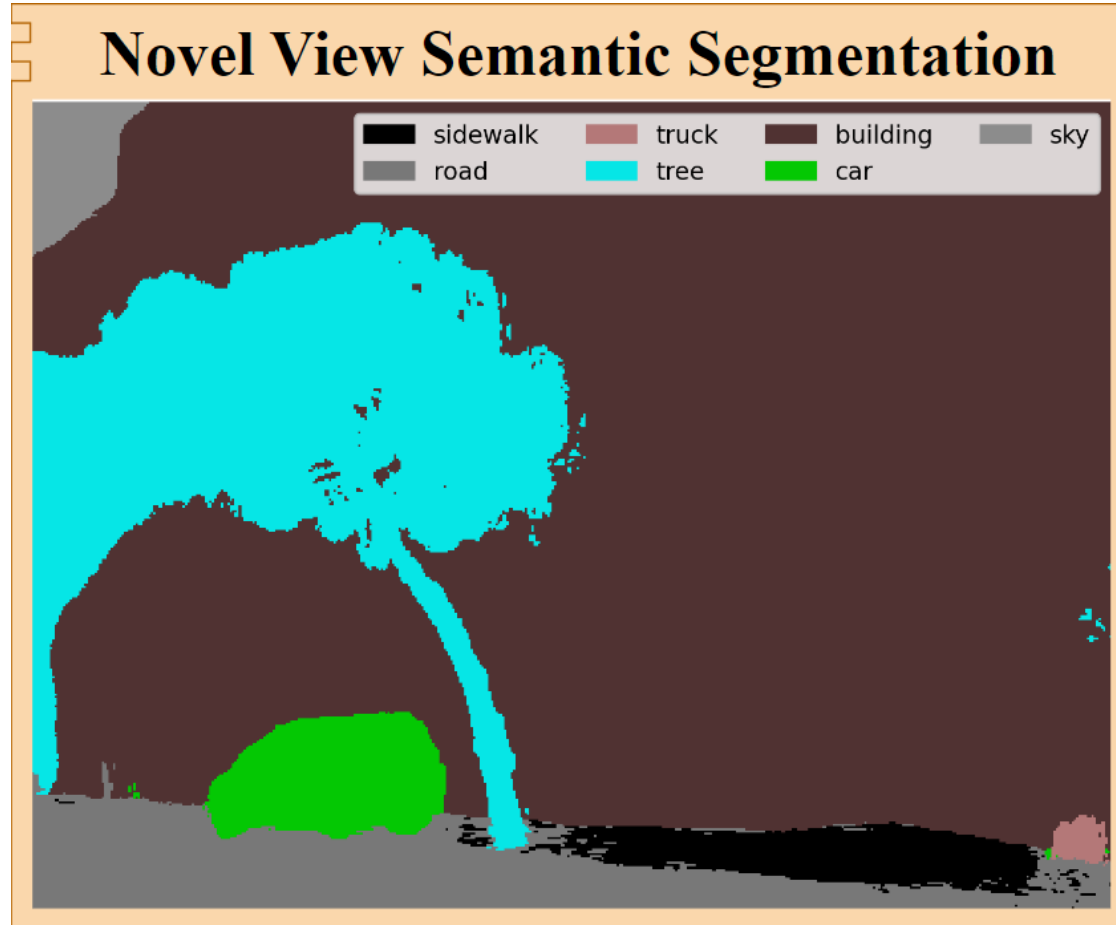


Radiance
Field



Feature
Field

Applications



Applications

Novel View Semantic Segmentation



Language-guided Editing



Applications

Novel View Semantic Segmentation



Promptable Segment Anything



Language-guid



Applications

Novel View Semantic Segmentation



Promptless Segment Everything



Promptable Segment Anything



Language-guided



Novel View Semantic Segmentation



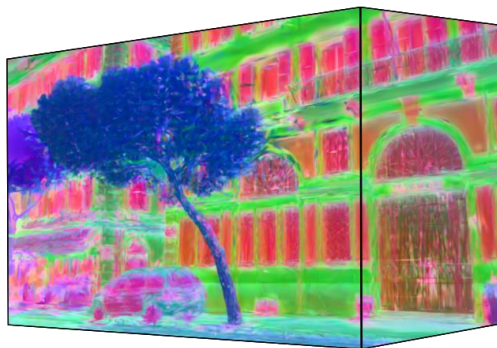
Applications



Promptable Segment Anything

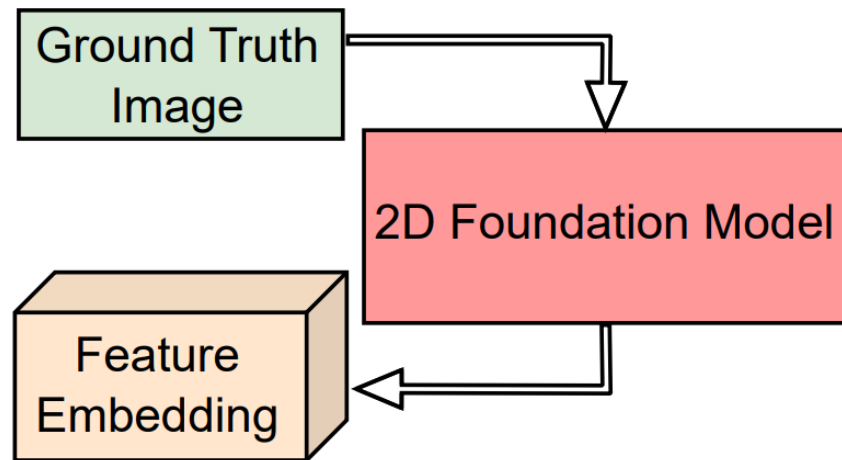
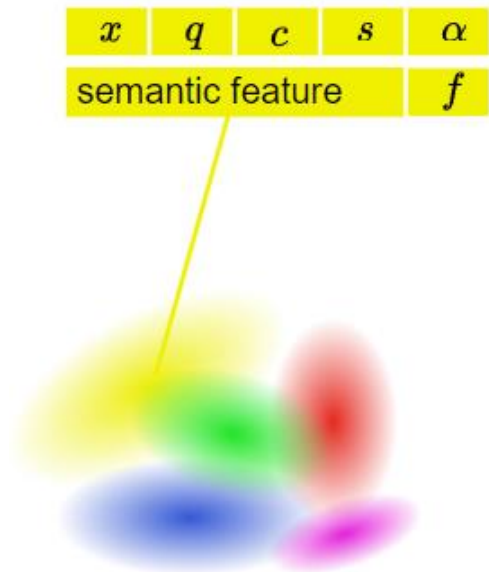


Language-guided Editing

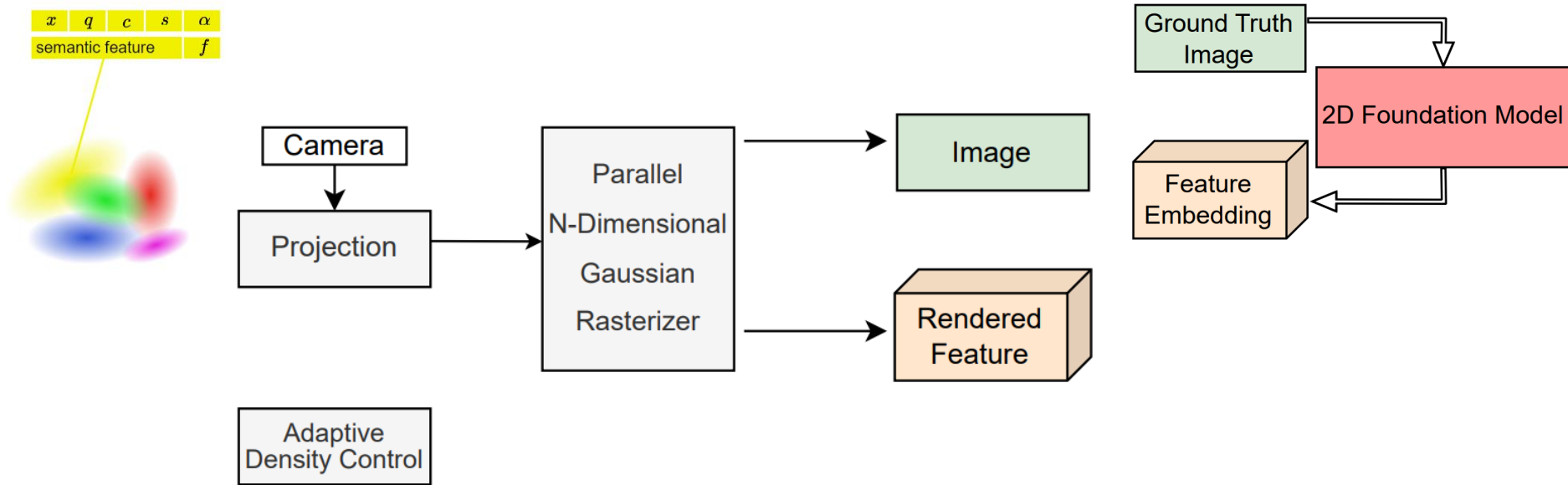


Promptless Segment Everything

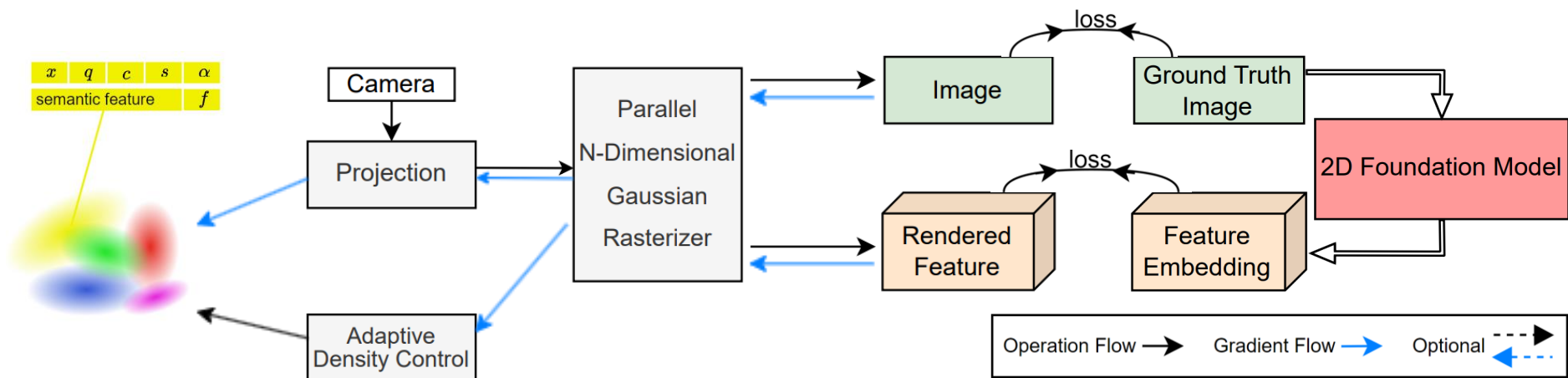




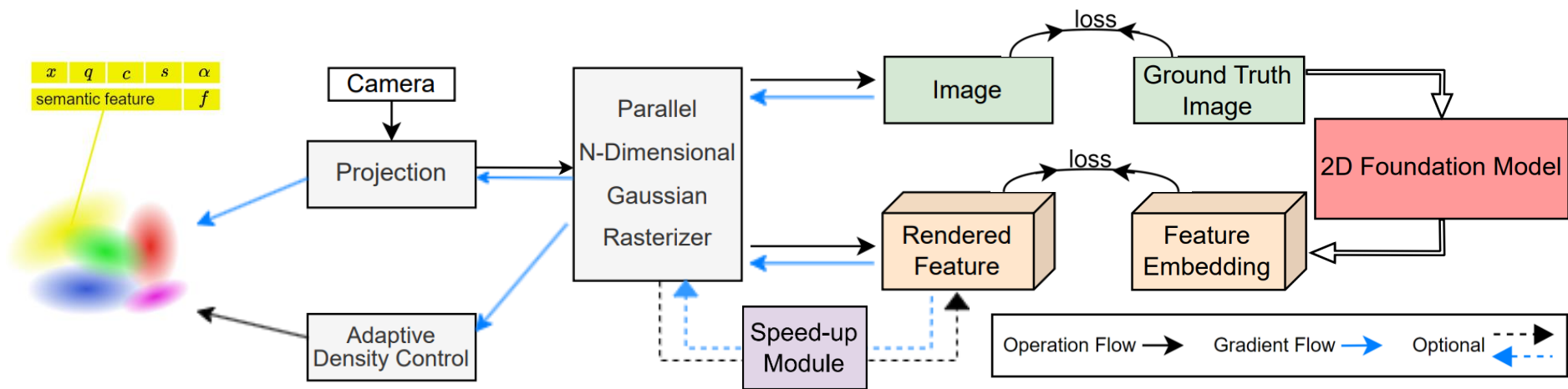
We utilize the state-of-the-art 2D foundation models as our teacher network.



We render the image and feature map via our Parallel N-Dimensional Gaussian Rasterizer.



CUDA empowered computation for end-to-end optimization.



Optional convolutional speed-up module for even faster training and inference speed.

Results and Demo